

Two Pointers & Sliding Window Study Guide

Two Pointers & Sliding Window Study Guide

1. Overview

The **Two Pointers** and **Sliding Window** techniques are among the most powerful patterns for array and string problems. They allow you to solve problems in **O(n)** time by eliminating unnecessary nested loops.

These patterns appear in hundreds of LeetCode problems involving:

- Searching
- Subarrays and substrings
- Sorting and deduplication
- Windowed constraints
- Optimization problems

Mastering these patterns is essential for efficient DSA problem-solving.

2. Two Pointers Pattern

Two pointers involve maintaining two indices that move through a sequence.

When to Use Two Pointers

- Array is **sorted**
- Need to find **pairs** or **triplets**
- Need to **reverse**, **partition**, or **deduplicate**
- Need to compare two sequences

Common Forms

- **Opposite-end pointers** ($l = 0, r = n-1$)
- **Fast/slow pointers** (cycle detection, middle of list)
- **Same-direction pointers** (merging, scanning)

Example: Two Sum II (Sorted)

```
def twoSum(numbers, target):
    l, r = 0, len(numbers)-1
    while l < r:
        s = numbers[l] + numbers[r]
        if s == target:
            return [l+1, r+1]
        if s < target:
            l += 1
        else:
            r -= 1
```

Example: Remove Duplicates from Sorted Array

```
write = 1
for read in range(1, len(nums)):
    if nums[read] != nums[read-1]:
        nums[write] = nums[read]
        write += 1
return write
```

3. Sliding Window Pattern

Sliding window is used for problems involving **contiguous subarrays/substrings**.

When to Use Sliding Window

- Subarray/substring problems
- Longest/shortest window satisfying a condition
- Counting or frequency constraints
- Windows with dynamic boundaries

Types of Sliding Windows

- **Fixed window** (size k)
- **Dynamic window** (expand/shrink based on constraints)
- **Monotonic window** (optimized with deque)

Template for Dynamic Sliding Window

```
l = 0
for r in range(len(s)):
    add s[r] to window

    while window is invalid:
        remove s[l] from window
        l += 1

    update answer
```

4. Classic Sliding Window Problems

4.1 Longest Substring Without Repeating Characters

```
def lengthOfLongestSubstring(s):
    seen = {}
    l = 0
    res = 0

    for r, ch in enumerate(s):
        if ch in seen and seen[ch] >= l:
            l = seen[ch] + 1
        seen[ch] = r
        res = max(res, r - l + 1)

    return res
```

4.2 Minimum Window Substring

```
def minWindow(s, t):
    need = Counter(t)
    have = {}
    required = len(need)
    formed = 0
    l = 0
    res = (inf, 0, 0)

    for r, ch in enumerate(s):
        have[ch] = have.get(ch, 0) + 1
        if ch in need and have[ch] == need[ch]:
            formed += 1

        while formed == required:
            if r - l + 1 < res[0]:
                res = (r - l + 1, l, r)
            have[s[l]] -= 1
            if s[l] in need and have[s[l]] < need[s[l]]:
                formed -= 1
            l += 1

    return "" if res[0] == inf else s[res[1]:res[2]+1]
```

5. Two Pointers vs Sliding Window

Pattern	When to Use	Example Problems
Two Pointers	Sorted arrays, pair/triplet search, deduplication	Two Sum II, 3Sum, Remove Duplicates
Sliding Window	Subarrays/substrings, constraints, longest/shortest windows	Longest Substring, Min Window, Sliding Window Maximum

6. Common Pitfalls

- Forgetting to shrink the window
- Incorrectly updating frequency maps
- Using two pointers on unsorted arrays
- Off-by-one errors in window boundaries

- Not handling duplicates correctly
-

7. Practice Roadmap

Beginner

- Move Zeroes
- Reverse String
- Maximum Average Subarray I

Intermediate

- Longest Substring Without Repeating Characters
- Fruits Into Baskets
- 3Sum

Advanced

- Minimum Window Substring
 - Sliding Window Maximum
 - Longest Repeating Character Replacement
-

8. Key Takeaways

- Two pointers and sliding window reduce $O(n^2)$ problems to $O(n)$.
 - Sliding window is ideal for contiguous subarray/substring problems.
 - Two pointers excel in sorted arrays and pair-search problems.
 - Mastering these patterns unlocks dozens of LeetCode problems.
-

End of document.